CPU Scheduling ready → run → no special timer mechanism

1. non-preemptive: process 主动 give it up when terminated / waiting I/O / 线程 yield statement

preemptive: ① Time Mechanism ② waiting/non-waiting give it up ③ priority — user / kernel mode

④ complete I/O or some special event

dispatch latency: 开始真 current process 中断 → kernel → next process (进入 OS, 准备信息 context switching)

preemptive: ③ ④ ⑤

non-preemptive: ① ② ④ → convoy effect: ① ④ ⑤

starvation: ② ③ ⑤

2. Scheduling Algorithm

① FCFS ② STF ③ SRT ④ PR ⑤ RR ⑥ Multi-level (Feedback) Queue

CPU utilization, Throughput, Turnaround Time (完成 - 到达), Waiting Time, Response Time

(六) Multi-Level Queue Scheduling 个 ready queue ⇒ 多 seperate queues

System: Priority Interactive: RR ⇒ response time

Batch: FCFS, SRT, STF ⇒ Turnaround Time

SRT, STF, 抢/非抢, 都自最优

average wait/turnaround

队列划分标准 (2种): ① Fixed Priority Scheduling ⇒ starvation ② Time slice

① 作业性质划分 ② 作业不会一直执行 high priority

(七) Multi-Level Feedback Queue process 可以在 queue 间 upgrade/downgrade

queue 内部 scheduling algorithm, queue 间也有 (上述 ①②)

Memory Management

1. CPU 可直接访问内存 (register, cache, MM)

2. Address Binding

(1) Compile time: process 的 address 在 确定有这个 process 时就被固定 VP = MP

(2) Load time: process 的 address 在 disk → MM 后, 被固定 VP = MP → VP = MP 不用 translate

(3) Run time: process 的 address 在 执行时, 都可以变化 VP ≠ MP 不是 disk

↳ OS + VM 实现的基础

→ 都经历过 Load, 所以 base + offset, process 大小不可超过 Limitation

3. Memory Allocation

low memory: Resident OS; high memory: User process

(1) Contiguous —— Fragmentation 碎片: 用 compaction 解决 MVT 的内部碎片

(2) Non-Contiguous

① Paging: PTBR: 放 base register, TLB: 快查表 ✗ 同时放 查物的 page/frame 编号

② Segment: external fragment

Virtual Memory

Demand Paging ⇒ FIFO, LRU, Optimal Anticipatory Paging ✗ 预取分页

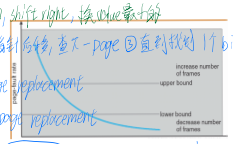
2种 trap { reference is invalid: terminal; page is not in MM: wait }

effective virtual access time = (1−f) × EAT + f × page fault service

EAT (effective access time) = t(1 + memory access time) + (1−t)(2 + memory access time) t = hit, f = not hit

Additional reference bit algorithm 利用固定时间, shift right, 换 value 最大的

Second chance algorithm ① 1 → 0 (二次机会) ② 找到 0 后转, 查下一 page ③ 直到找到 1 个 0 (二次机会) 换



Fixed allocation { Equal, Proportional } ⇒ Local page replacement

Variable allocation ⇒ Local & Global page replacement

Thrashing: Process are busy with doing swap page ⇒ CPU utilization ↓

内存利用率 due to 少量 frames ⇒ 大量 page fault I/O blocking ↑

进程数量 ② global replacement 会放大这种影响 Page fault frequency (PFF)

Working-set: 保证足量的 frame

Δ - 段 time 内, process exchange 总量

size: Δ 内, process 种类数 > frame size ⇒ 增加 frame

单 process PF ⇒ 增加 frame

若大部 process 需要 PF ⇒ 选择一部 suspend

FIFO: Belady's anomaly

Deadlock request → ready queue scheduling → run → release

Deadlock Characterization

① Mutual exclusion: 互斥, 同一时间, 只有 1 个 process 可用 resource

② Hold and Wait: 在等待 other resource 时, 手里不许有 resource ⇒ resource utilization ↓, starvation

③ no preemptive

④ Circular wait: priority order

4 methods to deal with deadlock

① Ostrich approach ② D prevention (预防)

③ D avoidance (提前分配 resource) ⇒ use additional infor

i 事先声明对每种 resource 的最大需求数

ii 动态检查资源分配后, system 是否仍处于 safe state

iii 资源分配 2 状态: 由可用资源数, 已分配以及资源数和 process 最大需求数共同决定

④ D detection: Process Termination: ① 几切 ② 1个1切 ⇒ starvation

(检查 + 恢复) Process Rollback: 重生后可能又走死在上一状态上去

如果 request edge 得到许可, 就会 reverse direction ⇒ assignment edge

清除有向边判断 deadlock ⇒ 从 Pi 入手, 若其 requesting edge 可以得到满足, 我们便 释放 Pi 的 both requesting/assignment edge

算法本质有向图 若存在 cycle, 拒绝 请求 一定无 cycle 才允许

Cooperating System (被) 影响有集合内其他 process

Synchronization 保证 cooperation 的有序进行 randomly choose/cicular queue

Critical Section Problem

① Mutex Exclusion: 互斥: 会发生 collision → 只有它有性, 不影响 其他程序

② Progress (CS 不能空着, 不该有 process 可进) → 证明: someone wants to enter, 其中有人能进

③ Bounded Waiting: 证明: 工作结束后, 一定可以进 (有限 time 内 = without forever)

int count = 0, int in = 0, int out = 0, semaphore S = 1	
Producer	***Consumer***

while (true) do {
// produce an item and store it
while (count == NUM_BUF) do { };
// wait for an unused buffer
P(S);
buffer[in] = nextProduced;
in = (in + 1) % NUM_BUF;
count = count + 1;
V(S);
}
	while (true) do {
	while (count == 0) do { };
	// wait for an available item
	P(S);
	nextConsumed = buffer[out];
	out = (out + 1) % NUM_BUF;
	count = count - 1;
	V(S);
	// consume a stored item
	}

P(S)
Disable Interrupt
S.value = S.value − 1
if (S.value < 0)
block (S.queue)
Enable interrupt

V(S)
Disable Interrupt
S.value = S.value + 1
if (S.value <= 0)
wakeup(S.queue)
Enable interrupt

while (true) do{
while (TestAndSet(lock)) do{};
<CS>
lock = false
<RS>}

boolean TestAndSet (U){
boolean returnvalue = U.2.
V = true;
return returnvalue}

monitor 收的像 semaphore 结构

外部的 single queue ⇒ P(mutex)

中间只允许 1个 process 执行 ⇒ CS

signal { Java: continuous → 主动 evet/wait; standard: wait → 唤醒 }

wait queue ⇒ conditional value

wait ⇒ V(); P()

signal ⇒ V(mutex)

File Concept → disk 上的 smallest unit: file 是 abstract data type (ADT)

1) file is a named collection of related information 有名字的相关信息集合

2) file infor - 一般在 directory structure 上,

3) file system: a collection of files with proper storage and directory structure

(4) 1个 disk 可有多个 partition (file system), 但 1个 partition 也可由多个 disk 组成

(5) attribute: Name, Identifier, Type, Location, Size, creator/owner, Protection, Time

(6) file operation: Create, Read, Write, Response, Delete, Truncate (清空)

Open ① 从目录结构中找到该文件 ② load from disk to MM; Close ① store from MM to disk

① file pointer ② file-open count ③ Disk location ④ access right

只有当最后 1个 process 用完关闭 file, 才得出计数 记录 file 绝地 owner 有权会结构目录开销太大

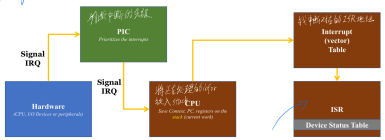
(7) 2 major types of files ① Program files: Source code, Object code, Execution Program

② Data file: Character or ASCII file, Binary file, Free-formatted file, Formatted or structured

③ { unstructured: 但单 word/bytes → Unix; structured: Simple record structure (固定 - 抽 - 抽), complex structure (html) }

Unix 的内核只认 bytes 字节流, 不支持 结构化 file, 但是由程序自己负责 (结构实现)

Magic Number → help with it



IRQ – Interrupt Request
PIC – Programmable Interrupt Controller (prioritizes interrupts, and ensures the CPU responds to the most critical ones first)
ISR – Interrupt Service Routine or Interrupt handler
PC – Program Counter

Access Method

Sequential Access: access in order, 支持 read/write next, reset, skip forward/backforward

Direct Access (relatively access), access directly, 支持 Read/Write n 用 Position n + Read/write next 定位

Index Access: 建立 seperate index file (包含 pointers to data block), data is stored in direct/relative file

Directory Structure —— 本身就 file, a collection of nodes containing infor of all files

manager file directory (Search, Create, Delete, List, Rename, Traverse 遍历)

1. Single-Level Directory: Name Problem (2 文件不可同名), Group Problem (无法文件分类)

2. Two-Level: MFD → UFD, seperate directory for each user, 不同 user 间可有 same name 文件

faster searching, access rights —— user 去新分到 入的 file

3. Tree-Structured: multiple levels, more efficient, grouping capacity 分类分组

current working directory: absolute or relative path names

4. Acyclic-Graph: 允许 1个 file/sub-dire 被多个 sub-dire 共享 (aliasing 别名)

Problem: ① double count ② Dangling Pointer —— 删 1个 link, file 也没了, 其他 link 变无效

Solution: reference count 创/删 link 时 +1/−1, 直到计数 ≥ 0, 才删 file

通过性质别名 (aliasing) 修改 file, file 同步更新

5. General-Graph: directing-link - directory 形成 cycle, reference count 失效

① garbage collection ② i 定期 检查 无效 并有 link/path ii 每次 添加新 link 都检查并拒绝

File Sharing —— ① multi-user system 多用户共同处理同一文件, protection scheme (认证)

② distributed system (use network)

③ remote file system (Manually (FTP, winscp), Automatically (J drive), 半自动 (web))

1. Client Server: 客户端挂载 mount 远程文件系统, 调用客户端文件调用接口, 远程执行并返回结果

2. Network-file system (NFS): Unix/Linux 系统下, 最常用的分布式文件系统 (Distributed File System)

3. Storage-Area Network (SAN): 大容量, 大容量, 高速

4. Cloud-Storage: Over the cloud:

File Storage Allocation

1. Linked Allocation: sequencial access 2. Indexed Allocation: random access

Storage 1. Secondary Storage 2. Disk Storage (local, remote) 3. Terbely 三级

① Host-Attached Storage (most standard arrangement) local, improve fault tolerance — mirror hard disk (RPID array)

② Network-Attached (NAS): remote procedure call Unix NFS - NAS based

③ Storage-Area Network (SAN): 当 file-sharing, private data network, special storage protocols

3. Primary: MM, 直接 CPU Secondary: hard disk/SSD, I/O bus Tertiary: CD-ROM, DVD

↳ hierarchical storage structure ① 用于 backup & archival 归档

② Robotic optical jukebox, collection of cartridge ③ offline storage: isolate 防火墙

Protection & Security → OS 的 code → rule

1. Protection: mechanisms & policy exi 修改 资源, 限制 访问 Internal OS

2. Security: 指 用户 (外部), 消息 (内部到外), malicious (恶意) 时意外的 introduction of flow (漏洞) to external OS

Problem: Intruders/crackers (入侵者) 企图 breach (突破) security

threat (威胁): 潜在安全 违反 (violation)

attack: 企图 breach (突破) security (可以是 accidental or malicious) 更好防护

Security Violation Categories

1. Breach of confidentially: 机密性破坏: 未授权读取 data (严重 危害)

2. Breach of integrity: 完整性破坏: 未授权 改动 data (严重 危害)

3. Breach of availability: 可用性破坏: 未授权 销毁 data (严重 危害)

4. Theft of service: 服务盗用: 未授权 使用 data (严重 危害)

⑤ Denial of service (DOS): 拒绝服务攻击, 阻止合法用户正常使用服务, 给系统 负担 同时 攻击 server

Security goal

Breaching Security 4种安全攻击方式

1. Masquerading (伪装成他): 假充合法用户, 窃取 权限

2. Relay attack (重放攻击): 把之前 录制的 包 发送 重新发送 server, 骗过 验证

3. Man-in-the-middle attack (中间人攻击): 在 两方 data flow 中, 冒充 sender or receiver (信息 篡改 "公开密钥中间人攻击")

4. Session hijacking (会话劫持): ex: 劫持已有 登录 状态 的 账户 登录 (登录)

Program Threats 5种 (恶意 破坏 安全 威胁): (code 层面的 恶意, malicious 威胁)

1. Trojan horse 木马: 隐藏 恶意 代码 伪装 成 合法 程序, 的 恶意 程序 "公开 恶意 程序" 的 机制 ex: 伪装 成 游戏 软件, 但 实 际 是 特定 (ex 窃 行 密码) 的 特洛伊 恶 意 程序

2. Trap door 后门: 开发 者 / 维护 者 设置 的 特殊 入口, 可以 绕过 正常 安全 检查 流程

3. Logic bomb 逻辑炸弹: 平时 正常 运行, 一旦 满足 特定 条件 (ex 某个 日期), 就 执行 恶 意 程序

4. Stack & buffer overflow 栈与缓冲区溢出, 利用 program code 漏洞, 写入 stack 外 以 超出 缓冲区 限制 的 data (覆盖 stack 上 的 内 存 数 据 返回地址), 调用 return, 跳转 到 恶意 代 码

5. Virus 病毒 嵌入 合法 程序 code 中 的 片段, 具有 自我 复制 能力, 破坏 目标 数据 甚 至 crash computer

通常 针对 特定 CPU 架构, OS, app, 常 以 传播 方式: email, macro

Security Measure level 4个 安全 防护 level

1. physical level ex: 机房 门禁 锁 与 devices

2. Human level ex: 员工 培训 防 社 骗

3. OS level ex: file 权限, process 隔离, bug fixed

4. Network level ex: 防止 通信 被 窃听, 防火墙

Computer Security Classification 安全分级

D: Minimal security C: discretionary protection 基于 策略 的 访问 保护

支持 用户 执行 特定 的 操作 用户 访问 控制 (访问 控制)

支持 用户 或 组 创建 的 操作 用户 访问 控制

B: 包含 C 的 全部 特性, 每个 object 都 有 安全 等级 标签 label 强制 访问 控制

A: 采用 形式化 设计 与 验证 技术 来 确保 安全 (最高 级)

Information Security CIA

confidentiality / Integrity / Availability

non-repudiation

不可否认性: 保证 了 行 此 者 会 被 记录

Cryptography ⟷ 与 cryptography 匹配

1. Transposition cipher 换位密码, 把字 重排, 把字 间 调换

2. Substitution cipher 用 替换表, 字母 替换

3. Vigenere cipher 维纳 密码

4. Caesar's cipher 每个字母 都 移固定 位数 ($A^2 = D, D^2 = B$)

Authentication

knowledge/Ownership/Inherence factor

2. (multi-) factor authentication

certificate, public-key infrastructure

① possession of private key 持有私钥 (公钥 加密 结果)

② 持有 权威 机构 签发 并 数字 签名 的 证书

Window Scheduling

Window 的 dispatcher 采用 preemptive priority-based RR 调度器 —— 32个 同 的 priorities

real time (16-31), user class (1-15), memory management (0)

6个 priority class 高级 REALTIME — IDLE - Priority Class

7个 relative classes TIME-CRITICAL — IDLE

用 完成 I/O 的 process 获得 的 值 时 立 即 提升 前 台 进程 (正在 用 的 窗口) 时 间 片 更 长 (比 后 台 进 程)

普通 process 默认 是 Normal - Priority - Class

Linux Scheduling

140个 priority class (100 real-time jobs, 40 others)

priority value 动态 调整 取决于 互 值 表 现: real-time (static), other jobs (dynamic)

初始 配额 → next process → all processes 用完 配额 → all 重置

Memory Management

1. Pentium Memory (IA-32) (at most 16,384 segments) { 2-level paging 4KB; 1-level paging 4MB }

2. ARM Memory VP = MP = 32 bit { 2-level paging 64KB; 1-level paging 1MB }

3. Itanium Memory VP = 48 bit, MP = 32 bit 4: 4KB 3: 2MB 2: 1GB

Virtual Memory

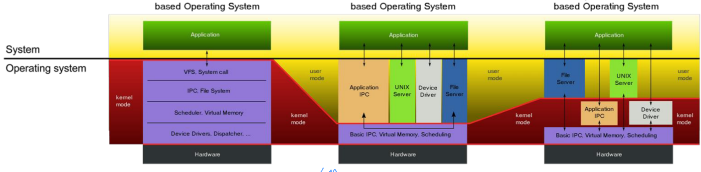
Window Virtual Memory —— demanding paging with clustering → anticipate surrounding page

发生 page fault: system 取 page 可能会 会 换 其 周 围 的

进 程 在 MM 中, 不可 保 working set minimum 1 page, 不可 高于 xx 个 page

若 系 统 内 存 阈 值 会 回 收 超 过 minimum 的 部 分

单 CPU 采用 advanced clock algorithm, 多 CPU 采用 FIFO



iCloud, dropbox, Google drive.

Linux Virtual Memory —— demand paging → valid/dirty/accessed bit
1个process 1个page table, 不同process可share MM
page finding 机制: kernel swap daemon thread; LRU aging mechanism

Window & *nix Deadlock 检查的overhead远不及deadlock出现频率
Ostrich approach: 直接ignore deadlock

File System → variation of linked allocation
1. DOS File System (FAT): All links are stored in seperate table 在disk开头
2. Window ↙ 继承
(1) HPFS: 可存入data & special attribute; FNODE, not first cluster as in FAT
(2) NTFS: journaling/recoverable file system by keep track of transaction ⇒ 小变复
Unix hard link; Linux access by open source NTFS server
3. Unix indexed allocation —— Fast file system
file infor 都在 inode中, 不是 file structure
4-level index block —— direct/indirect/double indirect/triple indirect block → large file
4. Linux —— Virtual File System 4 main object
① Superblock: global information ② Dentry: 每文件名和Inode的指针
③ Inode: meta file (每目录结构中内容), 并指向 block ④ File: actual opened file object, an Inode together with a current/writing offset; point to dentry

Synchronization
1. *nix
Message Passing: 普通进程间; Shared Memory: 同1进程中的多线程 Pthread库
Critical Section —— ① 用 mutex-locks ② Conditional variables
③ barrier 所有人到齐后, 再一起往下执行
2. Window
Critical Section —— ① 用 mutex/semaphore (special critical section) ④ barrier
② Conditional variables ③ 排列 wait function (synchronization objects)
Access Control —— access control list
1. *nix
(1) 单 process → 多 domain (2) /etc/passwd ✗ ⇒ authentication
(3) all objects in Unix are files (硬件也被抽象成 file)
(4) access control rwx —— owner/group/other
2. Window
(1) An item can be a file or a directory and has an owner. → can change control list, 此别人用
(2) 相较于 *nix 的 rwx, 还用了 D: delete, P: modify ACL → 直接剥夺所有权
owener/user 得到允许可改ACL D: make current account the new owner
(3) Packaging of access privileges into permission sets ④ full control
to simplify control: ① no access ② read: RX ③ change: RWXD ④ RWXDPO

Recently Development
(1) Microsoft: open source (2) Win 10: Windows Subsystem full Ubuntu kernel under
(3) Win 11: Powershell → Windows Terminal

1. Input/Output Direction M keyboard/screen → 改到想要的 (ex: file)
- There is a second output place for programs, called the standard error. It is normally used only for special system error messages and the standard error is also the screen.

> 清空原文件再写入 / 无原文件, 创建新文件再写入
>> 追加 $ GPA < grade2.txt (&) >> transcript.txt
- Input read from grade2.txt and output and error appended to transcript.txt

$ prog1 < infile | prog2 > outfile
- Effect like prog1 < infile > tempfile1; prog2 < tempfile1 > outfile (sequentially)

2. Pipe (concurrently) 在 Unix/Linux 中, 我们将 pipe 视为 I/O redirection, 但实际不同
Concurrency is about independent computations that can be executed in an arbitrary order with the same outcome. The opposite of concurrent is sequential, meaning that **sequential** computations depend on being executed step-by-step to produce correct results.

OS operation interrupt 在 OS 中相当重要
1. ① Dalmost all OS are interrupt-driven → OS 受中断驱动 (keyboard)
② interrupt allows OS to get control of system ex. 除数出现错误
* signal processing for interrupt { no response: terminate; yes: special response }
{ Pc <ctrl><Alt><Del>; Window <ctrl><c> } In Unix: clock algorithm — timer handler
2. interrupt security limitation 中断处理的安全问题
① 若 disabling interrupt (禁用中断), OS can't take back CPU
OS 不能干预 error (死循环), system 会 hang-up (ex: 蓝屏) → 防止用户滥用 interrupt-related 操作
3. ① hardware 的 privileged command, executed by OS & its core component
限制①, 保护 system 安全 security/stability → only be executed
4. dual-mode, ② 的具体实现 user 获取 OS 服务 'trap' 通一合法方式
user mode ⇒ normal (no privileged instruction) instruction | kernel mode ⇒ all instruction

System call user can't use privileged instruction
only OS processes could be executing in kernel mode

1. System Program
① OS ② Device Driver ③ Utility Program
④ Compiler ⑤ Assembler
2. Application Program
① Word Processor ② Spreadsheet
③ Web Server ④ Media Player ⑤ Email Client

3. 5 types
① File System: open(), close(), read()
② Process Control: fork(), exec()
③ Memory Management/Information maintenance brk(), mmap()
④ (Inter process) Communication: pipe(), socket()
⑤ Device Management: setConsoleMode()
user execute system call ⇒ trap
CPU switch ⇒ deal with the trap

① File manipulation ② Status infor ③ File Modification ④ Communication
⑤ Programming Language support ⑥ Program loading & executing ⑦ Utilities

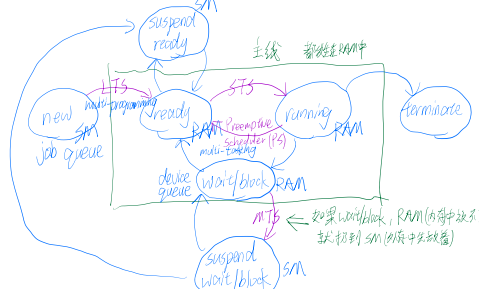
OS type & structure
1. ① Batch programming: 1任务 1任务 执行 Unix, Linux, Window Server
② Time-sharing: Interleaved manner by CPU, but each has time slice
③ Distributed: multiple-computers, network OS ⇒ { current: 单CPU; parallel: 多CPU; distributed: 多comp }
④ Multi-programming: Interleaved manner by CPU Unix
⑤ Real-time: deadline & penalty
2. *nix { Unix { BSD; System V by AT&T; Linux: LAMP development slot-arm } } Linux ⇒ Fastest supercomputer; Apache; MySQL; PHP Perl Python
3. Advantages of Unix & Linux: ① Reliability ② Security ③ Speed ④ Cost ⑤ Open Source

OS structure
1. Unix/Linux (Monolithic Kernel) ① 是一坨, 速度快 ② 耦合性强 ③ 体积大
2. Mac OS X (Microkernel) ① slow, IPC等信息沟通 ② modular 方便添加 ③ 只保证主要部分 Window
Advantages: ① easier to extend a microkernel
② easier to port OS to new architecture/hardware
③ More reliable (less code running in critical kernel mode)
④ More secure
Disadvantages: Performance overhead of user space to kernel space communication
3. Hybird Kernel —— Monolithic 与 Micro 的混合版 (Mac OS)
4. Modular OS —— 本质是 Monolithic, 但有 modular (Linux)
① Object-oriented approach | 1 fail may lead all fail
② Each core component is separated
③ Each talks to the others over known interfaces
④ Each is loadable as needed within the kernel

user — Application Programs — Shell — System Program — System Call — kernel — OS — HW

```
#!/bin/bash ("$@:1:4") ("$@:5")
teams=("$@") ← 将input存储成array
declare -A points ← 新array points
for t in "${team[@]}", do          字符串当下标
    points["$t"]=0  → 初始化 array points   字典
done ⇒ 每个sum=0
for file in round*.txt, do
    if [! -f "$file"]; then continue; fi;
    while IFS= read -r line || [-n "$line"]; do 每1行遍历
        read -r t1 t2 s1 s2 <<< "$line"  划分 → 匹配team name
        if [[ "${team[$t1]}" = ~ "$t1" || "${team[$t2]}" = ~ "$t2" ]]; then
            if ["$s1" -gt "$s2"]; then
                points["$t1"] = $((points["$t1"]+3))
                points["$t2"] = $((points["$t2"]+3))
            elif [ ]; then
            else
            fi
        fi
    done < "$file"
done
for t in "${team[@]}"; do
    echo "$t with ${points[$t]} points"
done output
```

1. 对于 ls -l 以可编辑形式复开 (no hidden)
接受参数 { -a 全部文件名 (包括隐藏文件); -al (全部 + 权限) }
① 类型 { - 文件; d 目录; l 软连接; b 块设备; c 字符设备 }
comp 1412 目录; lab1 可执行文件; link_to_file 符号链接; notes.txt 普通文件
file { symbolic link = 快捷方式; hard link = 副本 }
删除 file { symbolic link: 不会连; hard link: 还存在 }
② -o in cc ⇒ to specify the output filename
将源代码文件, 生成可执行的文件名 cc lab1.c -o lab1 ⇒ / lab1
③ alias (别名) 生成 ⇒ alias ls='ls -FC'
2. chmod bb4 filename ⇒ ① 文件 file; ② director
chmod g-r f
group 失去 read command
user, group, other r=4 w=2 x=1
⇒ 显示结果中, 若 filename 后有 *, 那则一定是 X 权限
3. grep 文件搜索 command
grep -i/-h/-l "text in file" filename
-i: ignore 大小写
-h: 不显示文件名, 只显示相关内容
-l: 只显示文件名
① ② ③
② 'polyu': 以 polyu 开头
'polyu$': 以 polyu 结尾
'poly.': . 可表示任何字符
[abc]: only one of a, b, c
[^abc]: 除了 a, b, c
'^polyu.$'
4. $wc filename ⇒ 4 可结果, 展示该 file 中 line, word, byte, character 数量
5. File Command
$ file filename
$ file filename1 filename2 filename3 ... ⇒ 查看 filenameX 的信息
$ lab*files=lab*.c
$ file $labfiles
$ usefiles="lab*.c *.doc *.pdf"
$ file $usefiles
⇒ 列出符合列出名字的 ___ 相关的文件



① SM → RAM: LTS, 从new中要符合筛选, 放入RAM中
RAM → RAM: STS, 几乎无任何筛选, 追求一个"快"字 (CPU switch)
RAM → SM: MTS, 几乎无任何筛选, 追求一个"快"字 (CPU switch)
↳ thrashing (抖动): 大量时间用于内存换页, 而非执行任务 尽可能多的将 process 放入 RAM
② Type of OS
(1) Multiprogramming: non-preemptive: 除非 process 自己不想干了, 尽可能多的将 process 放入 RAM
(2) Multi-tasking: preemptive: 其他更高级 process 抢了他的 CPU
③ Type of Process LTS makes good use of it
{ I/O-bound process: 长时间等待 I/O, 要插入短时间 CPU 使用
CPU-bound process: 长时间占用 CPU 短时间
↳ 进程的这2种 type 并非固定不变, 而是可切换, 动态变化

2. Cooperation, ① information sharing ② Computation speed-up ③ Modularity ④ Convenience
3. Terminated
(1) zombie process (僵尸子进程): 结束, 但父进程未调用 wait() 进行回收 子进程
(2) orphan process (孤儿): 父进程已结束, 但子进程还在继续
↳ 使用 Daemon Process (守护进程)
4. exec: path, NULL → execv; execlp: file, NULL → execvp; *argv[] → execv: *envp[]

一、Unix/Linux Interprocess Communication
↳ Shared Memory Mechanism
1. IPC Package set up Data Structure in Kernel Space
① 安全性与防御
② kernel 统一管理 all process
③ 不相关 process 也可通过 kernel 进行通信
1. Communication Method
(1) Direct Communication (like 邮箱)
(2) Indirect Communication (like mailbox)
2. Synchronization
Blocking 可以保证不会发生
① lost message ② waste resource: 非最精力看 message 是否到来
2. Share Memory is not practical in Unix/Linux
① 将 data 放入 address 可能会覆盖原 data, 导致 system crash
Protection Mechanism: OS 当中介, data 不直接访问 memory
② thread within the same process naturally share the memory space
User process use system call (Pthread) create shared memory segment

(2) Message Passing Mechanism
one direction
{ pipe: unidirectional, FIFO, unstructured (data stream) kernel 内置 buffer, 点对点, no bandwidth
socket: like named pipe (file) } ① 字节 byte 形式 ② byte flow
二、Communication Topology
① star topology (n-1), bottleneck 2 rounds ② Ring topology (n+1) many rounds
③ Linear topology (n+1), many rounds ④ Fully connected $\frac{n(n-1)}{2}$ fast communication
⑤ Tree topology (n-1), more rounds ⑥ Mesh topology $\frac{nn-1}{2}$ need routing, flexible
⑦ Hypercube: flexible → 2(n-1) ~ n(n-1) ⑧ Bus, not considered with pipes
三、(un)named pipe (本质上是1个file, 但一般用socket替代, not common in Unix/Linux Prog)
用 mkfifo 创建 named pipe, 实际是1个file, 不同process和找到这个file ⇒ heterogenous programs
named pipe is more flexible, while unnamed pipe is more secure
① only parent-child ② no name, can't find in system ③ dead after process dead
unnamed pipe 数量受限: 因为 fd 最大 1024, fd=0,1,2
Wildcard ⇒ normally used in ls
*: 代替任何1个/1组 字符
?: 代替任何1个字符
[]: 限制一个范围 [1-2] 中的1个字符 [0-2] [0-2A-Z] → [1-2] 无法匹配 12 而 {12} 可以
{}: ex: ls lab{1,12,12}.c 精确匹配这三个文件